

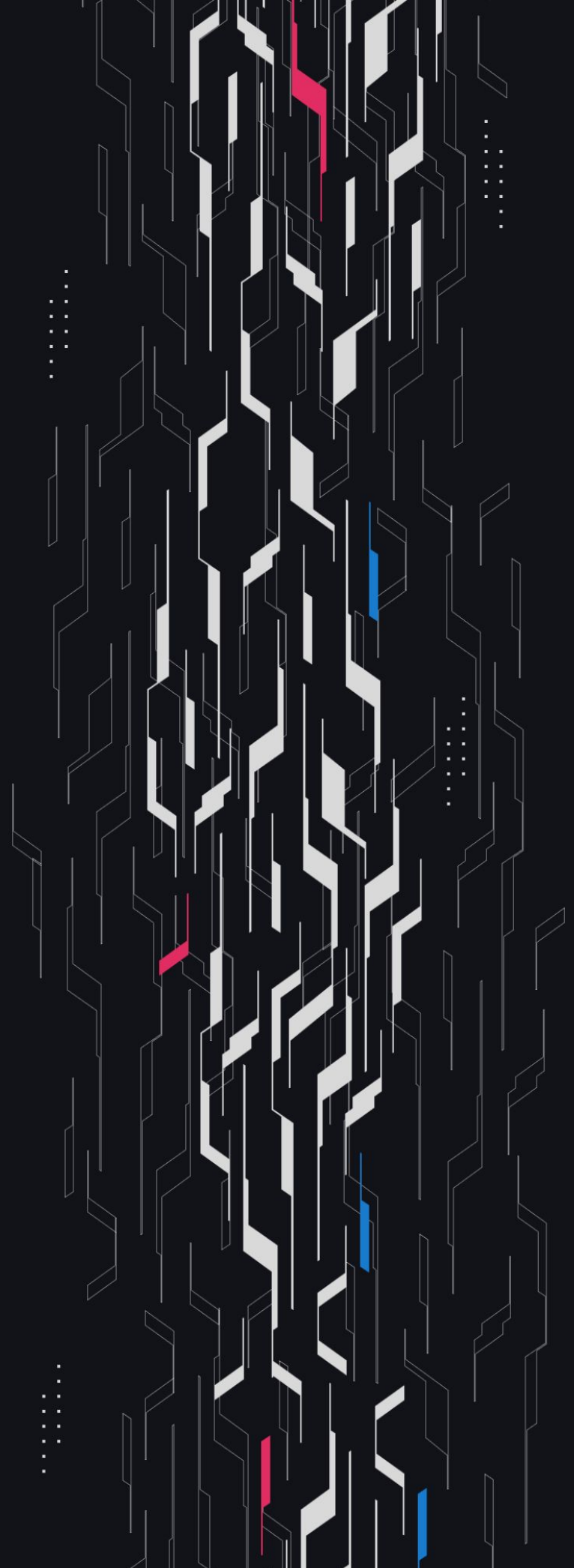
GA GUARDIAN

USDT0

**Polygon Deployment
and Peer Verification**

Security Assessment

September 27th, 2025



Summary

Audit Firm Guardian

Prepared By Curiousapple, Robert Reigada, Wafflemakr, Cosine, Osman Ozdemir

Client Firm USDT0

Final Report Date September 27, 2025

Audit Summary

USDT0 engaged Guardian to review the security of their USDT0's Polygon Deployment and Peer Verification. From the 7th of August to the 8th of August, a team of 5 auditors reviewed the source code in scope. All findings have been recorded in the following report.

Confidence Ranking

Given the lack of critical issues detected and minimal code changes following the main review, Guardian assigns a Confidence Ranking of 4 to the protocol. Guardian advises the protocol to consider periodic review with future changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

✓ Verify the authenticity of this report on Guardian's GitHub: <https://github.com/guardianaudits>

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only Low or Medium-severity issues were discovered. Design patterns are sound, and test coverage is reasonable. Small changes, such as modifying rounding logic, may introduce new vulnerabilities and should be carefully reviewed. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0 High/Critical findings. Varied Low/Medium severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1 High finding and ≥ 3 Medium. Varied Low severity findings.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities (~2/week). Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Information

Project Overview 5

Audit Scope & Methodology 7

Smart Contract Risk Assessment

Findings & Resolutions 10

Addendum

Disclaimer 22

About Guardian 23

Project Overview

Project Summary

Project Name	USDT0
Language	Solidity
Codebase	https://github.com/Everdawn-Labs/pos-portal https://github.com/maticnetwork/pos-portal https://github.com/Everdawn-Labs/usdt0-oft-contracts
Commit(s)	Playbook https://github.com/Everdawn-Labs/pos-portal Initial Commit: e6438f46f791381befb5d7d16528bed270cf43a4 Final Commit: e6438f46f791381befb5d7d16528bed270cf43a4 Scripts https://github.com/maticnetwork/pos-portal Initial Commit: 0b0a85abe9a6e7508bd35d9c495aa21ee5ab1678 Final Commit: 0b0a85abe9a6e7508bd35d9c495aa21ee5ab1678 Deployment https://github.com/Everdawn-Labs/usdt0-oft-contracts/ Initial Commit: 19d31b92745b95f52bd2a47979f0b72f16fab5c8 Final Commit: 19d31b92745b95f52bd2a47979f0b72f16fab5c8
Addresses	USDT0 - Proxy: 0xc2132D05D31c914a87C6611C10748AEb04B58e8F USDT0 - Implementation: 0x90040487a6c9f949c4f07cadcfb0f3b8eeab4229 USDT0 - Proxy Owner: 0x4DFF9b5b0143E642a3F63a5bcf2d1C328e600bf8 USDT0-OFT - OFT: 0x6BA10300f0DC58B7a1e4c0e41f5daBb7D7829e13 USDT0-OFT - Implementation: 0x8412d6553a2bd1b547bc48b86c3c5d897122ae47 USDT0-OFT - Proxy Admin: 0xa882c21C9df00958A958cde96f2B2Ae8FB4315B1

Project Overview

Audit Summary

Delivery Date	September 27, 2025
Audit Methodology	Static Analysis, Manual Review, Test Suite, Contract Fuzzing

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
● Critical	0	0	0	0	0	0
● High	0	0	0	0	0	0
● Medium	0	0	0	0	0	0
● Low	4	0	0	3	0	0
● Info	7	0	0	6	0	1

Audit Scope & Methodology

Addresses:

USDT0 - Proxy: 0xc2132D05D31c914a87C6611C10748AEb04B58e8F

USDT0 - Implementation: 0x90040487a6c9f949c4f07cadcfb0f3b8eeab4229

USDT0 - Proxy Owner: 0x4DFF9b5b0143E642a3F63a5bcf2d1C328e600bf8

USDT0-OFT - OFT: 0x6BA10300f0DC58B7a1e4c0e41f5daBb7D7829e13

USDT0-OFT - Implementation: 0x8412d6553a2bd1b547bc48b86c3c5d897122ae47

USDT0-OFT - Proxy Admin: 0xa882c21C9df00958A958cde96f2B2Ae8FB4315B1

Docs: <https://github.com/Everdawn-Labs/pos-portal/blob/playbook/playbook-usdt0.md>

Scripts: <https://github.com/maticnetwork/pos-portal/pull/187/files>

Transactions:

https://docs.google.com/spreadsheets/d/1yS43NI9ncC6XhnFpu2rqODYx_1cpeDvepVzngWWVf5k/edit?gid=0#gid=0

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: <i>High</i>	Impact: <i>Medium</i>	Impact: <i>Low</i>
Likelihood: <i>High</i>	● Critical	● High	● Medium
Likelihood: <i>Medium</i>	● High	● Medium	● Low
Likelihood: <i>Low</i>	● Medium	● Low	● Low

Impact

- High** Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.
- Medium** A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.
- Low** Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

- High** The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.
- Medium** An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.
- Low** Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.
Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Findings & Resolutions

ID	Title	Category	Severity	Status
L-01	Incorrect Pre-generated Calldata	Logical Error	● Low	Acknowledged
L-02	Avoid Using Private Keys	Best Practices	● Low	Acknowledged
L-03	Missing Block Validation	Validation	● Low	Acknowledged
L-04	Incorrect Role Revoked In Rollback	Logical Error	● Low	Acknowledged
I-01	Insolvency Risk For OFT Users	Logical Error	● Info	Resolved
I-02	Warning About Using Same Script	Informational	● Info	Acknowledged
I-03	Manual Script Steps Prone To Errors	Informational	● Info	Acknowledged
I-04	Double Check JSON Inputs	Configuration	● Info	Acknowledged
I-05	Mismatch Between Readme And Code	Informational	● Info	Acknowledged
I-06	Manager Actions Allowed After Migration	Informational	● Info	Acknowledged
I-07	Missing Migration Event	Events	● Info	Acknowledged

L-01 | Incorrect Pre-generated Calldata

Category	Severity	Location	Status
Logical Error	● Low	playbook-usdt0.md: 79-83	Acknowledged

Description

There is a rollback plan that aims to revert the proxies to the old implementation in case of a bug. This rollback plan uses pre-generated calldata.

While the pre-generated calldata for the ERC20Predicate uses the updateImplementation method to schedule and execute the rollback, the calldata for the RootChainManager uses the updateAndCall method with additional data to perform a grantRole operation after the rollback.

However, the grantRole function cannot be executed due to access control restrictions, and as a result, the current pre-generated calldata fails to deliver the rollback action in case a bug occurs.

Recommendation

The pre-generated rollback calldata for RootChainManager should be updated to call only the updateImplementation method.

Resolution

USDT0 Team: Pending.

L-02 | Avoid Using Private Keys

Category	Severity	Location	Status
Best Practices	● Low	UpdateImplementation.s.sol	Acknowledged

Description

The current preparation scripts optionally deploy new implementation contracts for RootChainManager, ERC20Predicate, and UChildUSDT.

To deploy these contracts, a PRIVATE_KEY must be specified in the .env file or via inline environment variable assignment. This approach risks exposing and compromising private keys.

Recommendation

Use a hardware wallet or encrypted keystore file.

Resolution

USDT0 Team: Acknowledged.

L-03 | Missing Block Validation

Category	Severity	Location	Status
Validation	● Low	RootChainManager.sol: 439	Acknowledged

Description

The `updateTokenMigrationStatus` will disable deposits and/or exits for a specific root token. Although the `UpdateTokenMigrationStatus` script validates non zero `lastExitBlockNumber` when disabling exits, this is not the case for the on chain validation.

Recommendation

Consider validating `lastExitBlockNumber > 0` in `updateTokenMigrationStatus` function.

Resolution

USDT0 Team: Pending.

L-04 | Incorrect Role Revoked In Rollback

Category	Severity	Location	Status
Logical Error	● Low	playbook-usdt0.md: 64-65	Acknowledged

Description

The USDT0 playbook includes rollback provisions intended for emergency scenarios. However, as per what documented, the `MIGRATION_MANAGER_ROLE` access is revoked from the zero address (0x0) rather than the actual address to which permissions were granted.

This oversight leaves the originally authorized address with unintended and unclear permissions.

Recommendation

Consider revoking permissions from the address that was originally granted `MIGRATION_MANAGER_ROLE` access.

Resolution

USDT0 Team: Pending.

I-01 | Insolvency Risk For OFT Users

Category	Severity	Location	Status
Logical Error	● Info	ERC20Predicate.sol: 113	Resolved

Description

The migrateBridgeFunds will transfer out USDT from the ERC20Predicate according to the collected USDT supply value on Polygon during the upgrade.

However, these funds will be sent to a Multisig address according to the playbook, instead of the the USDT OAdapter. Additionally, the playbook does not contain any extra steps regarding a subsequent deposit of USDT to another contract.

This will create an insolvency case, as there will be more USDT0 circulating in LayerZero chains than the amount of USDT available in the OAdapter to be redeemed.

Recommendation

After migrating USDT out of the ERC20Predicate, transfer them to the OAdapter contract, or set it as receiver in the encoded transfer call.

Resolution

USDT0 Team: Resolved.

I-02 | Warning About Using Same Script

Category	Severity	Location	Status
Informational	● Info	UpdateImplementation.s.sol: 96-101	Acknowledged

Description

According to the upgrade README, both the RootChainManager and ERC20Predicate implementation upgrades will be performed using the same UpdateImplementation.s.sol script.

The script can be run either by providing an input string or without any input. In the former case, it reads the necessary data from the provided input string; in the latter, it reads data from a previously prepared inputs.json file in the scripts/forge folder.

Since the inputs.json file can only contain one upgradImplementation key, and both upgrades use the same script, at least one of the upgrades must be performed using an input string, while the other can be read from the inputs.json file. The same case applies for UpdateTokenMigrationStatus script as well.

Recommendation

Consider using separate scripts for each upgrade so that all inputs can be prepared beforehand in the inputs.json file. Alternatively, modify _readInputs to read different keys from inputs.json based on whether the upgrade is for RootChainManager or ERC20Predicate.

Resolution

USDT0 Team: Pending.

I-03 | Manual Script Steps Prone To Errors

Category	Severity	Location	Status
Informational	● Info	disable-token-bridging/scripts/forge	Acknowledged

Description

The current playbook describes the migration steps for USDT0. These include generating calldata for creating multisig transactions, reading from a JSON file, copy and pasting results from console, testing bridge deposits and exits.

The amount of manual steps may lead to human errors, due to the amount of scripts to run, pasting the incorrect value in the JSON file or into the multisig UI for building the transaction.

Recommendation

Consider reducing the manual inputs as much as possible. Additionally, reduce the amount of scripts to run and/or use the forge script to write the output into the JSON file.

If possible, create the multisig transaction directly in the script, using the generated calldata. This avoids any UI phishing as you will be interacting with the smart contract directly.

Resolution

USDT0 Team: Pending.

I-04 | Double Check JSON Inputs

Category	Severity	Location	Status
Configuration	● Info	inputs.json.example	Acknowledged

Description

The preparation and migration steps involve using different foundry scripts that heavily rely on a JSON file.

Although the inputs.json.example is not the final JSON that will be used during migration, it already contains wrong addresses.

Recommendation

Due to the fact that scripts will heavily rely on this JSON file, extra care should be taken to verify that each value is correct.

Resolution

USDT0 Team: Pending.

I-05 | Mismatch Between Readme And Code

Category	Severity	Location	Status
Informational	● Info	RootChainManager.sol: 387-388	Acknowledged

Description

Step 8 of the playbook README mentions that exits should be blocked from the upgrade block: *"New plan is to just run the upgrade in block x, block exits from block x on (so no tx from x is valid exit) and manually check for any valid exits before the upgrade"*.

However, the code allows exits in the upgrade block and considers them valid Reference.

```
if (migrationStatus[rootToken].isExitDisabled
    blockNumber > migrationStatus[rootToken].lastExitBlockNumber) { // @note: if the
lastExitBlockNumber is the same as the current block number, exits are still allowed

    revert("RootChainManager: EXIT_DISABLED");
}
```

Recommendation

Update the code accordingly if the README is correct and the intention is to disallow exits in the upgrade block. However, note that this change would cause exits within the same block to fail and require manual intervention. Otherwise, update the README to reflect the current behavior.

Resolution

USDT0 Team: Pending.

I-06 | Manager Actions Allowed After Migration

Category	Severity	Location	Status
Informational	● Info	RootChainManager.sol: 473	Acknowledged

Description

The `MIGRATION_MANAGER_ROLE` can block deposit/exits and later `migrateBridgeFunds` to withdraw tokens from `ERC20Predicate`. Blocking deposits/exits does not have any validation, and migration is only allowed if both deposits and exits are blocked.

This means that manager may:

- call `migrateBridgeFunds` multiple times
- block deposits and exits to later migrate tokens, but re enable deposits or exits in the future again.

This might be unexpected, as migration should be a one time action.

Recommendation

Consider setting a state mapping that tracks migrated tokens, preventing manager to re-migrate or re-enable deposits/exits after migration.

If this is the expected behavior, document it in the `RootChainManager` contract.

Resolution

USDT0 Team: Pending.

I-07 | Missing Migration Event

Category	Severity	Location	Status
Events	● Info	RootChainManager.sol: 468	Acknowledged

Description

During funds migration flow, neither RootChainManager.migrateBridgeFunds or ERC20Predicate.migrateTokens emit an event.

This is a major state change that should emit relevant data, like it's currently done in the updateTokenMigrationStatus function.

Offchain services will likely need to inspect the migration transaction in order to read the USDT token receiver and amount withdrawn.

Recommendation

Consider emitting an event either in RootChainManager or ERC20Predicate, with the token receiver and amount.

Resolution

USDT0 Team: Acknowledged.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit <https://guardianaudits.com>

To view our audit portfolio, visit <https://github.com/guardianaudits>

To book an audit, message <https://t.me/guardianaudits>